CS 284: Programming Assignment 3: A GPS in Java

The internet, with its many online services, has truly revolutionized modern life. Among the many inventions like Wikipedia and YouTube, one slightly underappreciated one has been the invention of online maps.

Online maps have truly changed how we navigate through daily life. We can go to foreign cities with no preparation and still navigate around the area. We can get live updates on traffic or road closures.

Most importantly, though, online maps can give us the shortest route from where we are to a given destination. This lets us optimize travel times, and lets us avoid getting lost. But how do they find this shortest route?

In this homework, you will implement an algorithm for finding the shortest route in a given map. Along the way, you will also learn about how priority queues work in Java, and encounter some ideas about how to organize your projects.

Step 0: Getting Familiar With...

In this first section, we go over the basic algorithm you will implement, as well as the code we provide you to start with.

... The Way We Represent Maps

In order to be able to run algorithms on maps, we first need to somehow represent these maps in memory. More specifically, we need Objects that can store the structure of a map, including what connections are available.

We do this using the `Location` and `Connection` classes, which have been provided for you. As you read through this part, feel free to take a look at the actual classes involved so that you can better understand them.

You should not change these classes! You will use these classes in your code, but you must use them as-is.

The `Location` class represents some location we are considering. This could be something like a single building if we are looking at a map of a campus, or a city if we are looking at a map of the entire US. In each case, the `Location` stores both the identity of this location, as well as how it relates to others. In particular, the `Location` class has the following methods and constructors:

- The `Location` class has a single constructor, which takes in the name of this location. This creates a disconnected `Location` with the given name.
- The `getName()` method returns the name associated with this location. These will typically be unique in a given map.
- The `addConnection(Connection connection)` method adds a `Connection` to this `Location`. We cover `Connection`s below, but you can think of this as a road connecting the two locations.

  Note that this expects a `Connection` originating at this `Location`. If it does not start here, then the method throws an `EdgeConsistencyException`, which we use in cases where an operation does not make logical sense.
- The `getOutgoingConnections()` method returns all `Connection`s leaving this `Location`. This returns it in the form of an `Iterable<Connection>` (see below), with no guarantees about the ordering! You can only assume that it will return all valid `Connection`s, but you can't jump around between them, and can only read them in order.

On top of the `Location` class, we also have the `Connection` class, which represents some direct connection between two `Location`s. This can be a road, a highway, or something more complicated (you can later reuse this code for finding the shortest train route, for example). Each `Connection` has the folloiwng constructors and methods:

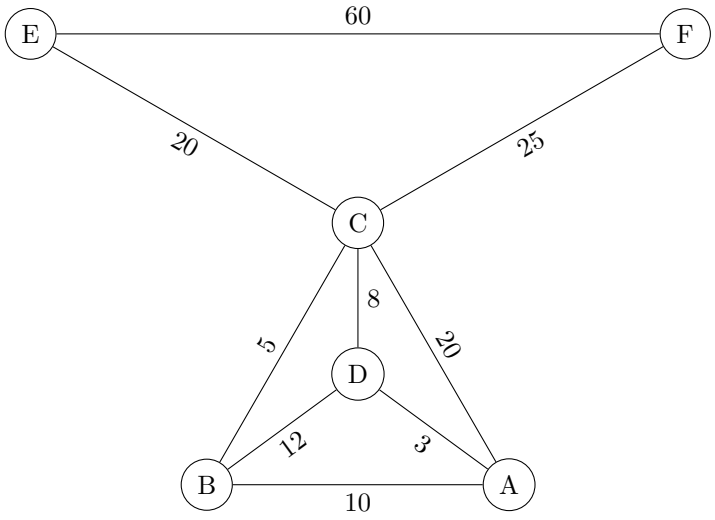
- The `Connection` class has a single constructor, which takes in the three values listed in the below methods.

- The `getStart()` method returns the `Location` this `Connection` originates from.
- The `getEnd()` method returns the `Location` this `Connection` ends at.
- The `getLength()` method returns the length of this `Connection`. Our goal in this program is to minimize the total length of all `Connection`s we take to get to our destination, thus finding the shortest path.

  Observe also that these values are immutable: Once a `Connection` has been made, there is no way of editing its fields. It simply exists as a convenient way of wrapping these pieces of information together.
- The `reverse()` returns the reversal of this `Connection`. As each `Connection` has a start and end, we provide this helper method to quickly get a `Connection` with the same length but pointing the other way.

Of particular note is this last notion of symmetry: For this problem, we will be assuming that all connections go both ways. So whenever we make a connection between two points, we also have a connection going back the other way, with the same length. There are no one-way streets, in other words. Note that our code doesn't enforce this! The `Connection` class is perfectly okay with allowing connections going only one way. However, it will be up to you to ensure that such symmetry is maintained when you later make your own maps of `Location`s.

Finally, we provide you with the `DummyLocations` class. By calling `DummyLocations.dummyLocations()`, you can generate a small map for testing. This will be useful when you just start developing, and we encourage you to play around a bit with it to start, to see how you can use the API of the `Connection` and `Location` classes. This small map encodes the following connections, with the given lengths:



... Iterables and Iterators

The previous section and the starter code both mention `Iterator`s and `Iterable`s. So what are those?

A very common use-case with any collection is iteration: Traversing every element in the collection and performing some operation on it. For example, you could try to print every element in a `List<String> lst` via the following loop:

```
for (int i = 0; i < lst.size(); i++) {
    String s = lst.get(i);
    System.out.println(s);
}
```

However, this is a lot of boilerplate for a simple idea: Just go over every element and print it. And this is also possibly inefficient: Repeated `get`s in a linked list each take linear time, for a total quadratic runtime!

Instead, Java provides a helpful construction known as a for-each loop:

```
for (String s : lst) {
    System.out.println(s);
}
```

The above loop does the same thing as the prior one, but is written much more condensedly.

Java allows you to iterate this way over any object which implements the `Iterable<T>` interface. This means that it must have an `iterator()` method, which returns an object of type `Iterator<T>`, which itself must provide a `hasNext()` and a `next()` method.

More specifically, the `Iterator` must provide the following functionality:

- The `Iterator` must somehow keep track of which element it is at in the object it is iterating over.
- The `next()` method returns the next item in the `Iterator`. In other words, this returns the item it is at while also updating which element it is at.
- The `hasNext()` method returns whether there is an element left in the `Iterator`, or whether we have reached the end of the underlying collection.

The earlier loop is then just code for the following:

```
Iterator<String> iter = lst.iterator();
while (iter.hasNext()) {
    String s = iter.next();
    System.out.println(s);
}
```

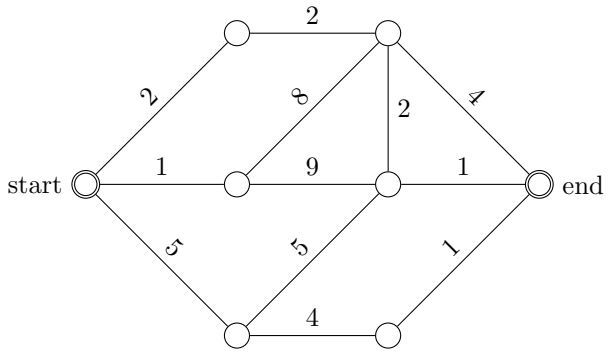
Note that this is often more efficient than doing repeated `get`s: Because the `Iterator` can maintain its internal state of where it is, it can more efficiently retrieve the next element. For example, a `LinkedList` might have an `Iterator` that stores a reference to a specific node, and can thus retrieve every node in constant time.

Many built-in collections already implement `Iterable`. Both `ArrayList` and `LinkedList`, `Stack`, `PriorityQueue`, and many others all have an `iterator()` method and can be used in a for-each loop.

In this homework, the `Location` class returns an `Iterable<Connection>` over its connections, and your `Path` class must implement `Iterable<Location>`, which will be relevant in Part 1.

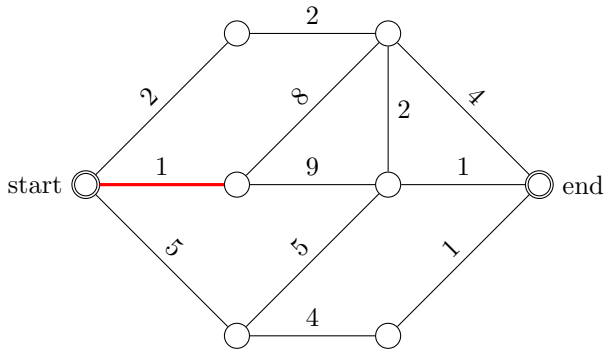
... Our Shortest Path Algorithm

So how do we actually find the shortest possible path? More specifically, given two points in our map, how do we determine what the shortest path between them is?



While we could blindly look over all possible paths, this is likely to be inefficient, as there can be very many paths. And a lot of these paths are going to be rather long, so we won't care about them.

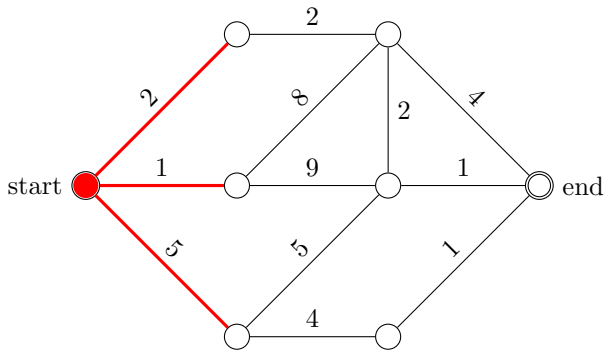
Instead, we can try to find the shortest path by finding the next-closest location. After all, if we want to minimize our distance, that seems promising.



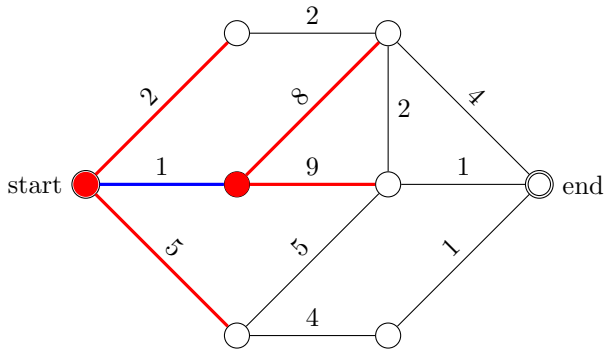
However, we don't necessarily want to commit. In fact, looking at this particular map, going down the closest location is a trap, as all the paths out of it are very slow. So if we continue down this path, we will be forced to take a needlessly long path.

Instead, we should keep our options open. More accurately, we should always strive to find the next-closest location out of all the ones we could have reached by this point.

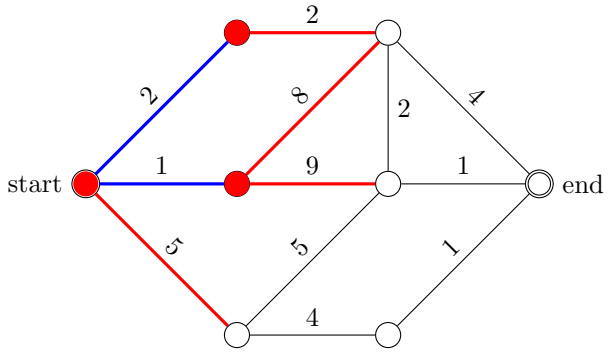
In particular, let us reframe our thinking. We start at the beginning, obviously. But from there, we have 3 options.



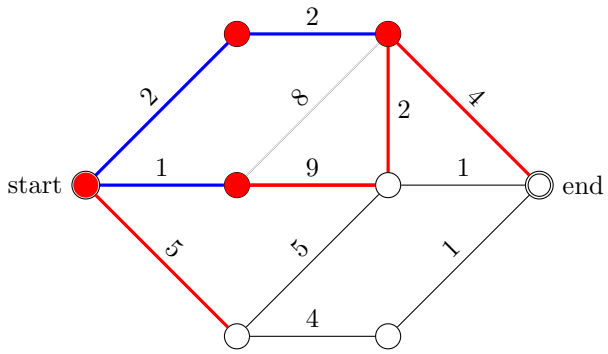
Out of these, the middle location is the closest, being 1 away from the start. So we can note down how we got there, and then consider all the places we can reach from it or our start.



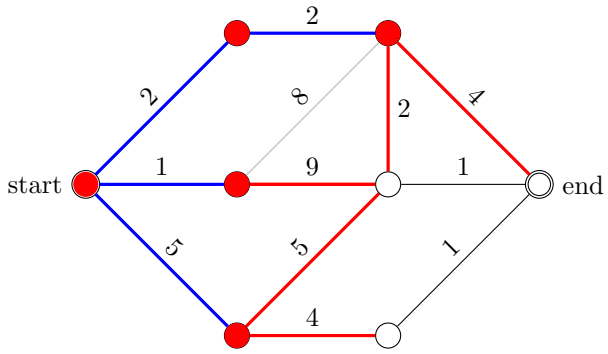
Now, out of the remaining locations, the top one is only 2 away from the start, so we can add it to our list of explored locations, and once again consider all the connections out.



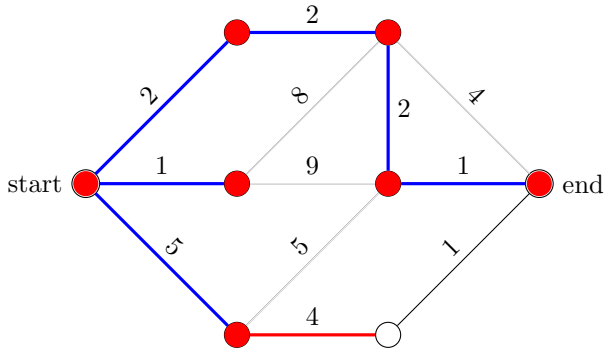
Now, the next location that is closest is the top right one. It is 4 $(2+2)$ away from our start, beating out any other location or path we have. Thus, we know that the best way to get there is via the top path, which we can note down. Note also that this means that the connection of length 8 is not useful to us, as both its endpoitns can be reached more efficiently by not using it.



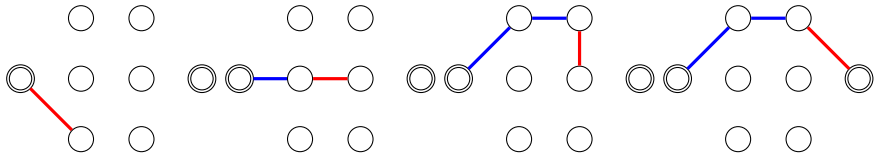
Now, we do see a path directly to the end... but we can't guarantee that this path is the shortest. So we keep going. Now, the closest point is the bottom left one, as it is 5 away. The middle right one is 6 $(2+2+2)$ away, so even though it's only 2 from its closest explored neighbour, it's further away from the start. And that distance is the one we care about.



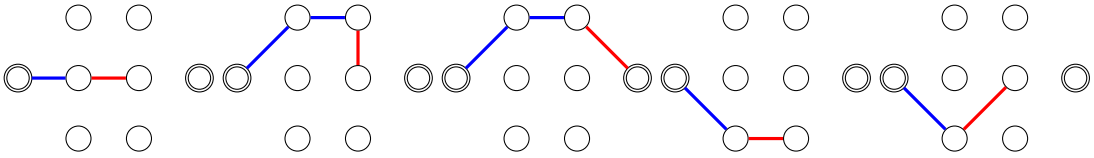
At this point, we will repeat this process twice more. First, we will take the center right point, and then finally, the closest point will be our target value. And at that point, we will have found the shortest path to the goal.



So let's break down this algorithm in more detail. The key concept is that we are at any point maintaining a frontier of possible ways of extending our paths. This is represented by the red edges in the diagram above: These are all the possible ways to get one step further away. But really, each of these is in essence a longer path. So more accurately, after we proccess the top right vertex, our frontier would consist of the following potential paths:



We then pick the shortest way of extending a path from those in our frontier. This is the next shortest path to get to a new location. This then expands our frontier by adding more potential paths starting at this new location.



We repeat this until our shortest path leads to the ending location.

This process thus requires our frontier to store paths, with two important properties: We want to be able to recover the current shortest path in the frontier, and to add new paths to the frontier. We already have a data structure that exactly does this: This is a perfect time to use a priority queue!

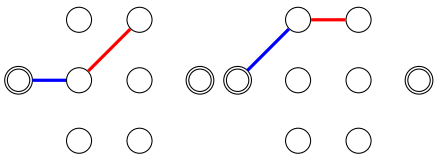
Thus, our alorithm starts to take shape:

1. Maintain a frontier of paths, using a priority queue.
2. Take the shortest path in the frontier.
3. If it ends at the target location, we are done.
4. Otherwise, find all possible ways of extending this path with one extra connection, and add all these new paths to the frontier.
5. Repeat steps 2 through 4 until we're done.

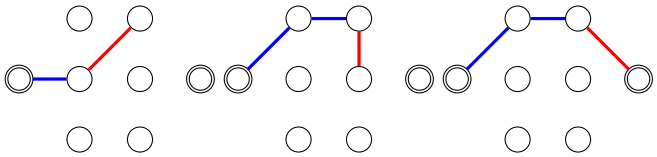
However, there are still a few extra details we have to iron out.

First of all, we need to decide how to start with our frontier. This is actually rather simple, as we have a very natural starting point: We can begin with a frontier that contains just one path, which in turn only contains the starting location. As all other paths start from here, starting with this 0-length path is the way to go.

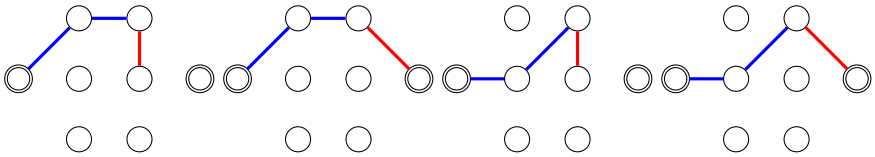
The second issue comes up in the case of paths converging. Consider a frontier that contains the following two paths:



Let's suppose that we decide to expand the latter path first, considering all ways of extending it. This will result in us adding several paths to the frontier.



Now, let's suppose that the first path in that list is the next shortest one. So that's the next one we should expand... or should we? After all, what would happen if we were to expand this path?



But observe: Of the two paths to the center right point, the top one is shorter. So any path built off of that one will be shorter than a similar path off of the bottom one. So the two new paths we've added are strictly worse to get to their destinations, so we have no reason to keep them.

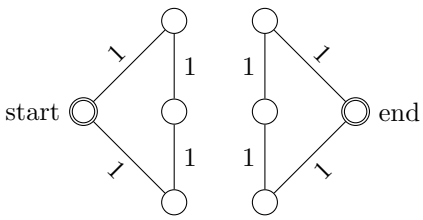
If a path ends in a location that we've already expanded at once, then expanding it again will only result in strictly worse paths. Thus, we don't want to expand again.

You can do this by having a `Set<Location>` of visited `Location`s. There are actually a number of ways to implement this, and we leave the exact implementation up to you. While we recommend having a set, you could also do this with a boolean array or a map. Still, we recommend starting your search with setting up an empty set using the following:

```
Set<Location> visited = new HashSet<>();
```

You should not change the `Location` class! You must use it as provided!

Finally, consider the following map:



We have no way of getting from the left side to the right side (perhaps there is a river of some sort with no bridge). But that means that if we keep expanding locations, we will eventually visit everything on the left side, and then run out of paths to expand. Thus, if we run out of paths to expand, we can conclude that there's no paths to the target, and should somehow indicate that in our output.

This thus gives us our general outline:

1. Create a priority queue to store our frontier, initially containing just the path starting and ending at the start location.
2. Retrieve the shortest path in the frontier.
3. If it ends at the target location, then we're done.
4. If it ends at a location that we've already visited, move on to the next shortest path.
5. Otherwise, mark this location as visited, and for each possible way of extending this path, add the extended path to the frontier.
6. Repeat steps 2 through 5 until we find the shortest path to the target or until the frontier has no more paths left.
7. If our frontier runs out, then there is no way of getting to the target from the start, so we should communicate that in whatever we return.

And that's our entire algorithm! However, each of these steps is deceptively complicated. Thus, the rest of this handout goes through this one step at a time, indicating how you should approach this.

Step 1: A Class To Represent Paths

Before we can even begin to implement the algorithm, we first need some way of representing a path. While we could just use an array or list of `Location`s, there are enough restrictions on what a path can be that it makes sense to actually have a dedicated class.

Thus, our first step is to implement a `Path` class. This represents a path, a sequence of `Location`s starting at the starting point of our search.

Our `Path` class needs several pieces of functionality:

- It needs to store the entire path. We can't just store the endpoints of the path, as we will need to retrieve all the intermediate locations as well.
- As we will want to find the shortest path, it makes sense to implement the `Path` class to efficiently return the length.
- Likewise, we need to be able to quickly find the endpoint of this path, so that we can know where to extend from.
- Finally, we need to have functionality to extend a path, by adding another connection to the end of it.

In order to accomplish this, you need to implement the following methods in your `Path` class:

- You will need to have a constructor. As every `Path` must contain a starting `Location`, you will have the constructor accept the `Location` that you start from.

- You must have a `getLength()` method. This should return the length of the path, the sum of the lengths of all the connections in this path.
- You must have a `getEnd()` method. This returns the endpoint of the `Path`, the last `Location` it reaches.
- You must have an `extend(Connection connection)` method. This creates a new `Path`, one that is equivalent to the current one extended with the extra `Connection` added to the end. In practice, this will extend the `Path` by one `Loation`.

  As we want to potentially extend the same `Path` multiple times, this must return a copy and leave the original unchanged.
- Your `Path` class should implement the `Iterable<Location>` interface. This means it should have an `iterator()` method which lets you go through all the `Location`s in this `Path`.

Once you have this, you now have the ability to represent paths, and can move on to trying to build the algorithm.

To summarize, your `Path` class should have the following basic framework:

```
public class Path implements Iterable<Location> {
    public Path(Location start) { /* ... */ }
    public Path extend(Connection connection) { /* ... */ }
    public double getLength() { /* ... */ }
    public Location getEnd() { /* ... */ }
    public Iterator<Location> iterator() { /* ... */ }
}
```

Hints and Suggestions

In order to help make development smoother, we provide a few tips and tricks for how to implement this task.

- Test as you develop! As you add new methods to your code, doublecheck that they work. Write test code, or just mess around in a `main` method, to ensure that everything is working as intended. It's easier to catch bugs early, than to write everything in one go and then struggle to find where a problem is coming from.
- While the `getLength()` method can theoretially recompute the length of the `Path` each time, you can probably improve performance by manually storing it. This will simplify your `getLength` method, although you will now need to maintain it as you change your `Path`.
- Note that if you are saving the length, it turns out that you don't actually need to store each `Connection`. You can just store only the `Location`s in the path.
- In order to help the copying process of `extend()` go more smoothly, you may want to include a helper constructor. It is up to you how you design this, but we recommend having this extra constructor to more neatly organize your code.
- Depending on how you store your `Location`s, you might be able to avoid having to write a custom `Iterator` for the `iterator()` method. In fact, the official solution implements the `iterator()` method in just one single line, using only a single call. Think about how you can reuse code that others have already written for you.
- What happens if we have a `Path` that ends at some `Location` A, and we try to extend it with a `Connection` from B to C? Well, that isn't really a meaningful operation, when you think about it! In all likelihood, this situation should not come up when properly using the `Path`, and we will not be testing for what happens on such invalid inputs. However, you may want consider using the `EdgeConsistencyException` here as well.
- Don't forget to add Javadoc comments to everything! It's a lot easier to write comments as you go, rather than having to backtrack at the end.

Step 2: Comparing Paths

Now that you have implemented the `Path` class, you can try to create a `PriorityQueue<Path>`, in order to build our algorithm. However, if you try that, you will quickly find that it gives an error.

See, in order to have a priority queue, I need to be able to compare my elements. And unfortunately, Java has no idea how to compare two `Path`s, as they are seemingly unrelated `Object`s from its perspective.

So let's change that!

If you check the documentation for the `PriorityQueue` class, you will see that it has multiple constructors. And the important one for us is the option to take in a `Comparator` class.

A `Comparator<E>`, as the name might imply, is a class designed to compare two objects of type `E`. More specifically, it defines a method `compare`, which takes in two objects and returns which of the two is "smaller" according to some ordering. It does this by returning a number: A negative number means that the first object is "smaller", a positive number means that the second one is "smaller", and a 0 means that they are "equal".

Note that I put "smaller" in quotes for a reason. A `Comparator` has its own rules for what counts as smaller, and you can have multiple `Comparator`s for the same type but with different rules. For example, you could make a `Comparator<String>` that orders the strings lexicographically (in dictionary order) or which just orders the strings from shortest to longest. In each case, it would return a number such that if the first number is earlier in its corresponding ordering, then it's negative, and if the second is earlier, then it's positive.

Note that such a `Comparator<E>`, if defined in a consistent way (so that no triangles are formed), fully specifies an ordering on the set of possible `E`. And that means that any collection of `E` now will have a smallest element. So by creating a `PriorityQueue<E>` using a `Comparator<E>`, we can have the items in the priority queue be ordered according to the ordering of the `Comparator`.

So that is your next step: Implement a `Comparator<Path>`, so that we can have a `PriorityQueue<Path>` work. This should be a nested class called `PathComparator`, which we have already provided for you, which needs to properly implement the `Comparator<Path>` interface. In particular, note that we want the `PriorityQueue` to output the shortest `Path`, so you should base how the `compare` method works off of that.

Hints and Suggestions

- The difficulty of this step comes from understanding how a `Comparator` works. Once you actually understand, you should not overthink the implementation.
- There is actually a one line implementation of the `compare` method, although you don't have to find it, and a slightly more verbose one also works.
- If your `PriorityQueue<Path>` doesn't seem to be working right, and it's refusing to output the shortest `Path`, check if your `Comparator` is not working backwards: It might be putting longer `Path`s earlier, which is an easy mistake to make.

Step 3: Finding the Shortest Path

Now that we have all our helper classes set up, we can set up our main class.

The rest of your work for this homework will be done in a `StevensMaps` class. This class will only contain `static` methods, and represents a hypothetical app for handling all the map operations we would want.

To start with, we want to be able to find the shortest path, so we need to implement the shortest path algorithm here. More specifically, we want a `shortestpath(Location start, Location target)` method, which returns the shortest path from `start` to `target`. If there is more than one, it can return any of them, and if there are none, it should return `null`.

More specifically, let us revisit the exact algorithm you need to implement here.

1. Create a priority queue to store our frontier, initially containing just the path starting and ending at the start location.
2. Retrieve the shortest path in the frontier.
3. If it ends at the target location, then we're done.

4. If it ends at a location that we've already visited, move on to the next shortest path.
5. Otherwise, mark this location as visited, and for each possible way of extending this path, add the extended path to the frontier.
6. Repeat steps 2 through 5 until we find the shortest path to the target or until the frontier has no more paths left.
7. If our frontier runs out, then there is no way of getting to the target from the start, so we should communicate that in whatever we return.

Now, however, we can fill out the details. And we can use the methods in our helper classes to implement each step. For example, to create our frontier, we can now use our `Comparator` to make a `PriorityQueue`. In fact, every step can now be neatly expressed using just the methods we have implemented earlier! By having implemented all the steps separately, building the algorithm just requires putting the pieces together!

Hints and Suggestions

- Most of the heavy lifting has been done earlier. In fact, every step of this algorithm can theoretically be implemented in about two lines given all the methods we already have.
- It might help to review your loop controls like `continue` and `break`.
- Note that as soon as you find any path to the target location, it is automatically the shortest one. Thus, in step 6, you only need to ensure that you repeat the process until the frontier has no more paths in it.
- In order to test this code, it helps to have a test case whose answers you know. As such, for the example map given in `DummyLocations`, the following are some shortest paths, along with their lengths:

  1. From A to C: A to D to C, total length 11.
  2. From E to F: E to C to F, total length 45.
  3. From A to E: A to D to C to E, total length 31.

  You should test your code to ensure that it outputs these paths.

Step 4: Reading Maps From Files

By this point, you should have tested your code quite a bit. And if you tried making your own maps, you probably have gotten tired of manually setting up all the `Location`s and `Connection`s.

Thus, our next step is to be able to efficiently set up such maps. For this, we want to be able to write all the places and connections neatly in a file, and to then load them into our program. Which is what you will do in this step: You will implement a `parseFile(File file)` function, which takes in a `File`, and outputs a `List<Location>` based on the contents of that file.

We will specify these maps using a standard file format. Such a file consists of 4 parts:

1. A number n, which specifies the number of locations in this map.
2. n names of locations.
3. A number m, which specifies the number of connections in this map.
4. m connections, specified as a pair of locations and a length.

The numbers n and m are both integers stored on their own line. n will always be a positive integer, and m will always be a non-negative integer.

The n locations are stored on separate lines. In other words, the n lines after the number n each contain the name of one location each.

The encoding of the connections is the most complicated part here. Each connection is given as a triple of three numbers: Two non-negative integers and one non-negative real-valued number (for example `0 2 20.0`). The real-valued number is the length of this connection. The numbers, however, specify the indices of its two endpoints. In particular, the n locations are assumed to form a 0-indexed list, and each connections specifies its endpoints by giving where they lie in that list.

For example, consider the following, found in the file `test-input-graph.txt` we give to you:

```
6
Arts and Sciences Centre
Business School
City Center
Department of Computer Science
Entrance to Tunnel
Fairgrounds
9
0 1 10.0
0 2 20.0
0 3 3.0
1 2 5.0
1 3 12.0
2 3 8.0
2 4 20.0
2 5 25.0
4 5 60.0
```

Here we have 6 locations, with the same names as our earlier test locations. We also have 9 connections. The first connection is between location #0 (the Arts and Sciences Centre) and location #1 (the Business School), with a length of 10.0. In fact, by comparing the rest of these connections, they exactly encode the connections given in `DummyLocations`! For example, the second-to-last connection connects locations #2 and #5 (the City Center and Fairgrounds), with a length of 25.0.

Your `parseFile` function, which must live in your `StevensMaps` class, must take in a file in this format, read it, and output a `List<Location>` representing the locations in it (with all the appropriate `Connection`s).

If the file is found to not meet the proper specifications (such as the file not existing, the numbers n or m not being in the right range, or a connection having invalid endpoints), then you should return `null`, to indicate that this `File` can't properly be parsed into a map.

Hints and Suggestions

- You may find the `Scanner` class incredibly useful. Notably, you can create a `Scanner` from a file, and then read individual words. You are encouraged to read up on how `Scanner`s work.
- The provided `test-input-graph.txt` file should let you test your parser, because it should give the same map as `DummyLocations`. In particular, after you parse it, you should get the same shortest paths as you did with `Dummy Locations` earlier.
- Typically, we discourage using an `ArrayList` due to resizing being an issue. However, observe that in this case, you actually know the amount of `Location`s you will have right from the start, as you read in n on the first line. Thus, you can actually create an `ArrayList` with enough space from the get-go, thus avoiding having to resize it.
- Remember to avoid leaving any dangling newlines in your `Location` names!

Step 5: Putting It All Together

Now that we finally have all the infrastructure in place, we can put it all together and write a user-facing program.

We want a program that can load in arbitrary maps, and then get the shortest paths between arbitrary locations. To accomplish this, we now introduce a second type of file: The task file.

A task file specifies a map to use, followed by a series of origin-destination pairs. We want our program to take in such a file, and to then output the shortest path for each such pair.

Each task file has the following format:

- The name of a map file, containing which map these tasks are defined over.
- A positive integer T, specifying how many tasks this file will contain.
- T tasks: Pairs of names of locations from the map file.

For example, take a look at the provided `test-input-tasks.txt` file:

```
test-input-graph.txt
3
Arts and Sciences Centre
City Center
Entrance to Tunnel
Fairgrounds
Arts and Sciences Centre
Entrance to Tunnel
```

The first line specifies that these tasks are defined over the map given in `test-input-graph.txt`. In particular, all the later locations are going to be locations in this map. This is then followed by specifying that T is 3, and giving three tasks:

1. Find the shortest path from the Arts and Sciences Centre to City Center.
2. Find the shortest path from the Entrance to Tunnel to the Fairgrounds.
3. Find the shortest path from the Arts and Sciences Centre to the Entrance to Tunnel.

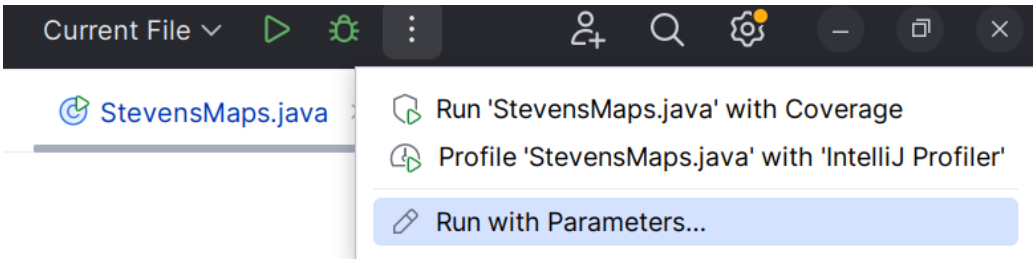
Your job is to implement, in your `main` function in `StevensMaps`, a program that takes in such a task file, and runs all the tasks specified within.

More specifically, your program will be given this file as a command-line argument. When running a Java program, you can specify some inputs as part of the instruction of running the command.

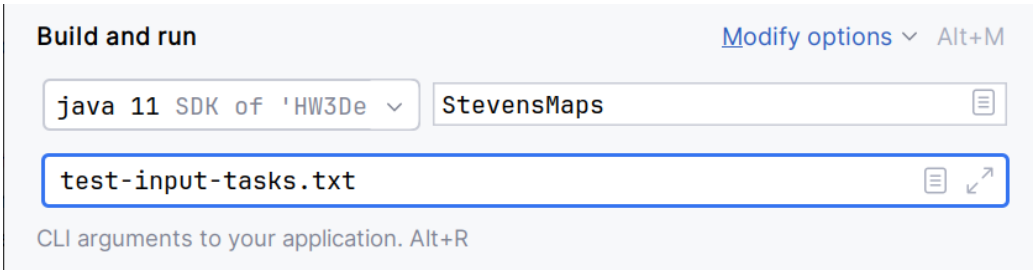
If you run it from the command-line, you specify the argument just by appending it to the end of the command.

```
java StevensMap test-input-tasks.txt
```

If you are using an IDE, however, you probably have to find some option to run it "with parameters". For example, in IntelliJ IDEA, you have to go click the three dots besides the run button to be able to specify the arguments.



Once there, you can specify the arguments that you intend to pass in.



These command-line arguments can be accessed in your program through the use of the `args` parameter to `main`. In particular, as you will pass in just one file name, you can find it in `args[0]`.

Once you have the file, you must then open the corresponding map file (yes, you have to open a file given by another file), and then parse that map. From there, for each given task, you must then output the following:

- The distance from one end to another along this shortest path.
- The cities you find along said path.

For example, if you run the program on the provided `test-input-tasks.txt` file, the output should be the following:

```
Distance from Arts and Sciences Centre to City Center: 11.000000
Locations Along Path:
    Arts and Sciences Centre
    Department of Computer Science
    City Center
Distance from Entrance to Tunnel to Fairgrounds: 45.000000
Locations Along Path:
    Entrance to Tunnel
    City Center
    Fairgrounds
Distance from Arts and Sciences Centre to Entrance to Tunnel: 31.000000
Locations Along Path:
    Arts and Sciences Centre
    Department of Computer Science
    City Center
    Entrance to Tunnel
```

Once you have this, you have a working program, that lets you calculate shortest paths on arbitrary maps. We then encourage you to test your code some more on some additional maps and task files. We have provided a few for you, but encourage you to also make your own for testing.

Hints and Suggestions

- Note that the task file specifies the location names, and not their indices. So to find the `Location` object corresponding to each name, you will have to look through your `List<Location>` returned by the parser. We recommend writing a helper function for this.
- When your program runs, it might run into several possible erroneous states:
  - What if the specified map file does not exist?
  - What if a start or endpoint is not actually in the map?
  - What if there isn't actually a path from the start to the endpoint?

  In each case, you should output a convenient message to the user. Note that if one path is not valid, you should still check and run the other tasks, rather than immediately giving up on the entire file.
- You are likely to run into a very odd bug where your first task runs perfectly fine, but your later tasks all seem to fail or output wrong paths. To realize what happens here, we encourage you to use the debugging capability of your IDE to trace through your code, and to consider carefully which locations have and haven't been visited at any point in time.
- In general, using the debugging tool can be very helpful for this assignment, as it lets you see what things are like in memory at any given point in time.
- Remember to include Javadocs on all your classes and methods!

Additional Reading

This section is not necessary for implementing this assignment, but provides additional context for people who wish to learn more.

The algorithm you implemented in this homework is known as Dijkstra's Algorithm, and is considered one of the best algorithms for finding shortest paths in graphs.

This algorithm is known to take time $O(m + n \log n)$, when used with a proper implementation of a priority queue.

It can also be proven to always return the shortest path in a graph. This can be proven by showing using induction that the first time we process a vertex, we have reached that vertex using the shortest path there. Then, because we keep taking the shortest path out of our frontier, if we hit a new vertex, we must have found the shortest path to get there, maintaining the invariant and getting the desired performance.

If all the length of edges are the same, you can actually get better performance by using a queue, resulting in an algorithm known as Breadth-First Search. This is often used for simpler cases (such as if you are looking at a subway map and the distances between any two adjacent stations is always the same), but does not work when the edges are weighted.

There are some other algorithms that accomplish similar tasks, which we briefly mention here. If you have to compute many pairs of shortest paths, there are some other algorithms that are often used, including the Floyd-Warshall algorithm. And if the graph has negative edge lengths, then Dijkstra doesn't work directly. This material and more is covered in more detail in CS 385, but if this piques your interest, I encourage you to read up on it early.

Submission Instructions

For this homework, you must submit your `Path` and `StevensMaps` classes. This should include your `Path.java` file, your `StevensMaps.java` file. You should not submit your `Location.java` and `Connection.java` files, as you should not change them from what you are given. We will be using the same `Location` and `Connection` classes that we have provided you for testing, so make sure not to change their functionality!

Note that each class must be located in the `pa3` package, or we will be unable to compile it otherwise.

Submissions must be handled through Gradescope (accessible through the homework page on Canvas). Grades will be released through Gradescope once the homework is graded.

Rubric

Category	**Description**	**Points**
Part 1: Paths	Correct implementation of the methods in `Path`	20
Part 2: Comparators	Correct implementation of a `Comparator<Path>`	15
Part 3: Shortest Path	Correct implementaiton of `shortestPath`	30
Part 4: Map Parsing	Correct implementaiton of `parseFile`	20
Part 5: Stevens Maps	Correct implementation of a `main`	15
Total		100